The Hong Kong Polytechnic University

Department of Electronic and Information Engineering

EIE2111 Lab 7: Command-line Processing and Linked List

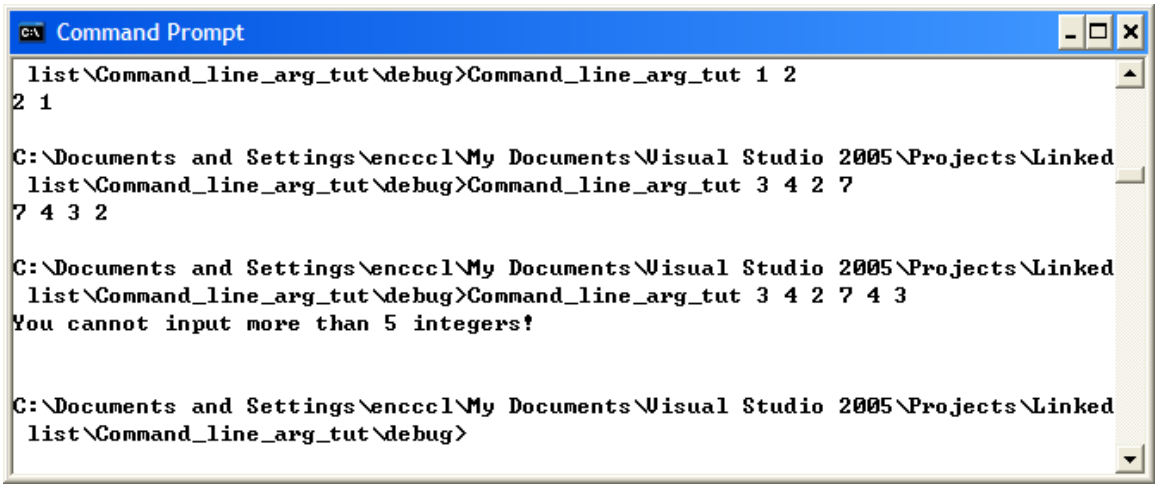
(Deadline for Submission: Check the course information)

Introduction

This laboratory exercise is designed to give you hand-on experience in Command-Line Processing and Linked List.

Question 1

Write a program that uses command-line processing to get a number of integers and then print them out in descending order. The number of input integers is limited to 5; otherwise, an error message should be displayed. A sample output is shown below:



Question 2

You have learned the concept of singly linked lists with different operations. In this question, you are required to implement different operations in a doubly linked list. The following is the definition of the class of a doubly linked list:

```
class Doubly_linked_list // Use a class Doubly_linked_list to represent an object
{
    public:
    // constructor initialize the nextPtr
    Doubly_linked_list()
    {
        prevPtr = 0; // point to null at the beginning
        nextPtr = 0; // point to null at the beginning
    }
    // get a number
    int GetNum()
    {
        return number;
    }
    // set a number
    void SetNum(int num)
    {
        number = num;
    }
    // get the prev pointer
    Doubly_linked_list *GetPrev()
    {
        return prevPtr;
    }
    // set the prev pointer
    void SetPrev(Doubly_linked_list *ptr)
    {
        prevPtr = ptr;
    }
    // get the next pointer
    Doubly_linked_list *GetNext()
    {
        return nextPtr;
    }
    // set the next pointer
    void SetNext(Doubly_linked_list *ptr)
    {
        nextPtr = ptr;
    }
private:
    int number;
    Doubly_linked_list  *prevPtr;
    Doubly_linked_list  *nextPtr;
};
```

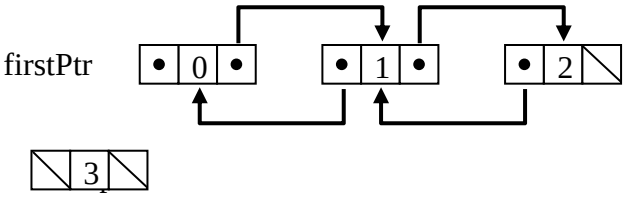
You are required to write the following functions:

1. Create a doubly linked list. The integer assigned in a node of the list is its sequence (i.e., 0 is assigned in the first node, 1 is assigned in the second node, and so on.). Remember to assign the previous and next pointers properly.

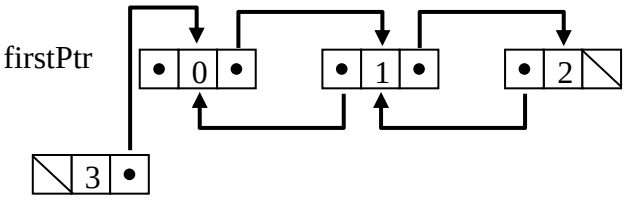
2. Print a list.

3. Print a list ***reversely***.

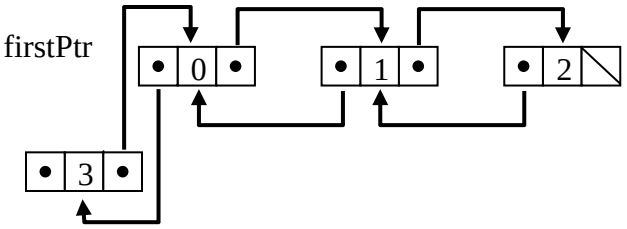
4. Insert a node at the front of a list
   Diagram:
   Before



   Step 1

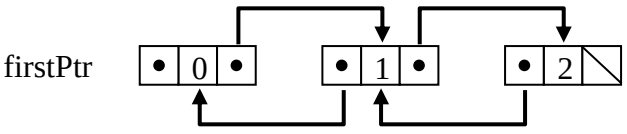


   After

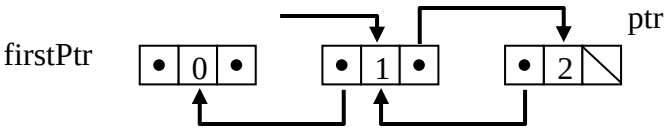


   return ptr

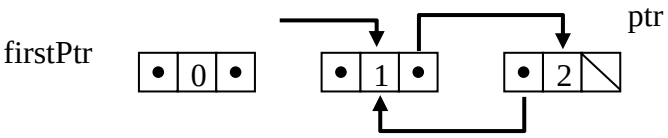
5. Remove a node from the front of a list
   Diagram:
   Before



   Step 1



After



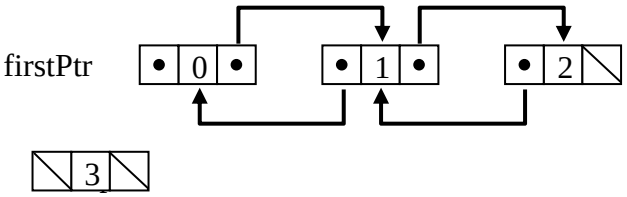
delete firstPtr

return ptr

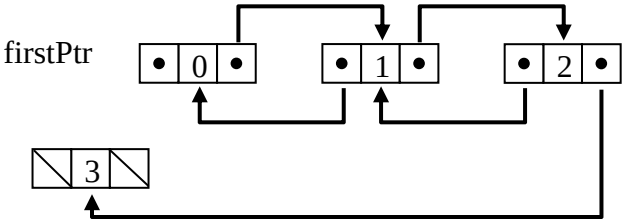
6. Insert a node at the back of a list

Diagram:

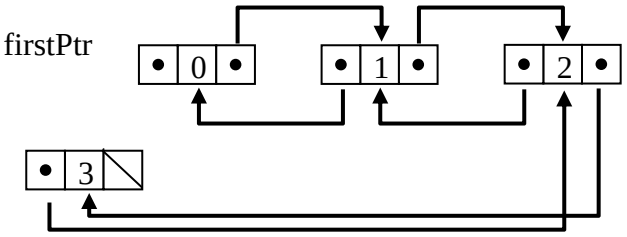
Before



Step 1



After

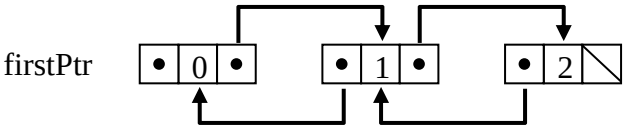


return ptr

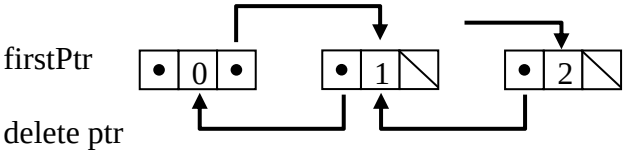
7. Remove a node from the back of a list

Diagram:

Before



After



delete ptr

return firstPtr

8. Insert a node before the node with a given value in a list

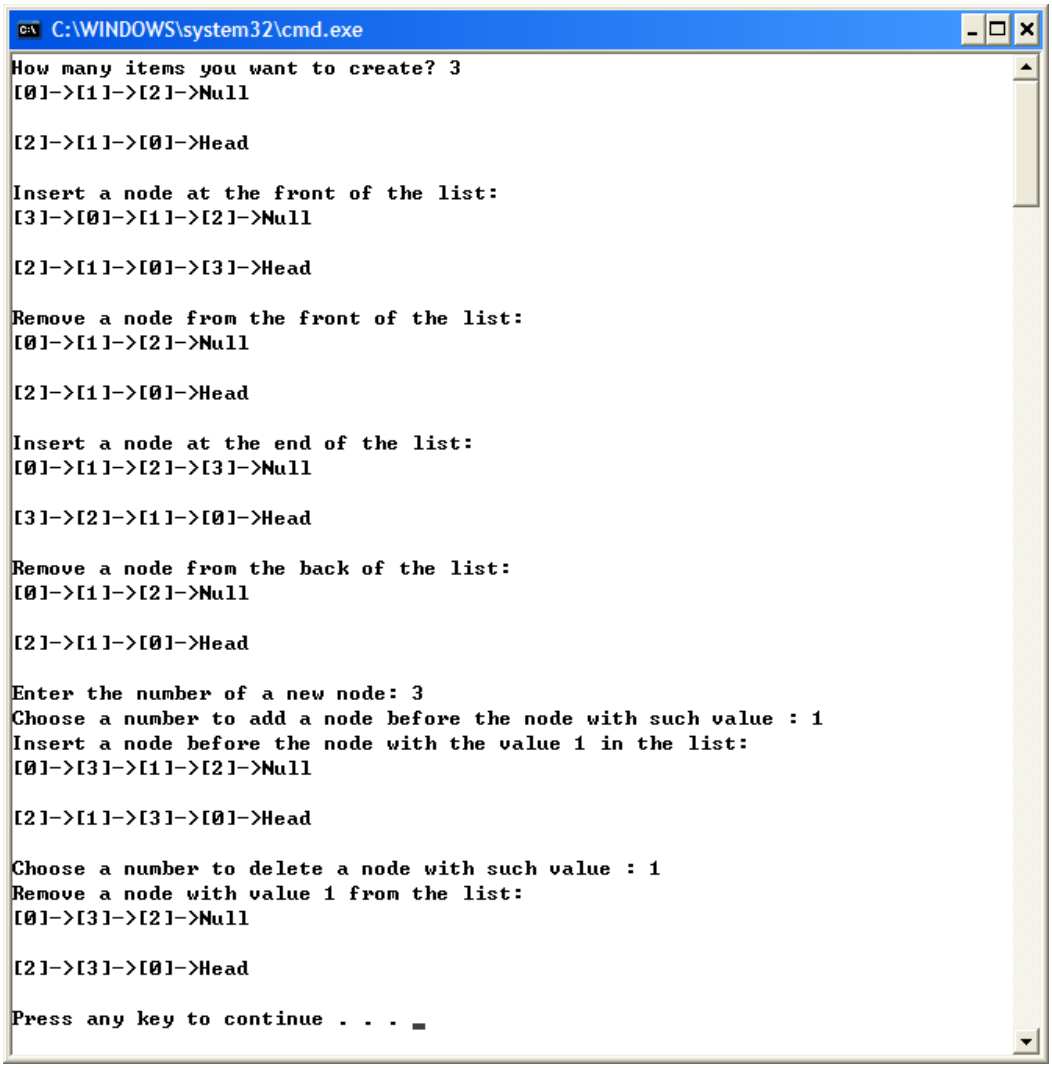
9. Remove a node with a give value in a list

You should use the following main program to test the above nine functions:

```
int main()
{
    int num;
    Doubly_linked_list  *headPtr;
    Doubly_linked_list  *newPtr;
    cout << "How many items you want to create? ";
    cin >> num;
    headPtr = Create_Doubly_linked_list(num);
    Print_Doubly_linked_list(headPtr);
    Print_Doubly_linked_list_reversely(headPtr);
    newPtr = new Doubly_linked_list;
    newPtr->SetNum(num);
    cout << "Insert a node at the front of the list:" << endl;
    headPtr = Insert_node_at_front(newPtr, headPtr);
    Print_Doubly_linked_list(headPtr);
    Print_Doubly_linked_list_reversely(headPtr);
    cout << "Remove a node from the front of the list:" << endl;
    headPtr = Remove_node_at_front(headPtr);
    Print_Doubly_linked_list(headPtr);
    Print_Doubly_linked_list_reversely(headPtr);
    newPtr = new Doubly_linked_list;
    newPtr->SetNum(num);
    cout << "Insert a node at the end of the list:" << endl;
    headPtr = Insert_node_at_back(newPtr, headPtr);
    Print_Doubly_linked_list(headPtr);
    Print_Doubly_linked_list_reversely(headPtr);
    cout << "Remove a node from the back of the list:" << endl;
    headPtr = Remove_node_at_back(headPtr);
    Print_Doubly_linked_list(headPtr);
    Print_Doubly_linked_list_reversely(headPtr);
    cout << "Enter the number of a new node: ";
    cin >> num;
    newPtr = new Doubly_linked_list;
    newPtr->SetNum(num);
    cout << "Choose a number to add a node before the node with such value : ";
    cin >> num;
    cout << "Insert a node before the node with the value " << num << " in the list:"
<< endl;
    headPtr = Insert_node(num, newPtr, headPtr);
    Print_Doubly_linked_list(headPtr);
    Print_Doubly_linked_list_reversely(headPtr);
    cout << "Choose a number to delete a node with such value : ";
    cin >> num;
    cout << "Remove a node with value " << num << " from the list:" << endl;
    headPtr = Remove_node(num, headPtr);
    Print_Doubly_linked_list(headPtr);
    Print_Doubly_linked_list_reversely(headPtr);

    return 0;
}
```

The sample output is listed below:



Instructions

a. You are required to submit your C++ programs (the whole projects created in Microsoft Visual Studio) in Question 1 and 2 to Blackboard. Zip all of them into a single file.
b. The deadline of the submission: **Check the course information**.

Lawrence Cheung
January 2021